

Transformaciones XSLT y XSL-FO



Jose Antonio Alcalá Hinojosa

Índice

Índice.....	2
Introducción.....	3
XSLT.....	4
1. Creación del Documento.....	4
2. Creación de la plantilla.....	4
3. Añadir elementos HTML.....	5
4. Añadir estilos “CSS”.....	6
5. Comprobamos resultados.....	7
XSL-FO.....	8
1. Creación del documento.....	8
2. Creación de la plantilla.....	8
3. Añadir elementos de formato.....	9
4. Comprobamos resultados.....	10

Introducción

En este documento, daré una explicación sobre cómo transformar un documento XML en otro tipo de archivo o incluso en un PDF para impresión. Aquí encontrarás paso por paso todo lo que hay que hacer para llegar a esa transformación y que tu código XML sea más legible y atractivo para personas externas.

Mi código XML recopila información sobre una biblioteca de videojuegos en Steam, donde se recopila el título del videojuego, su desarrollador, año de lanzamiento, género, plataforma en la que se puede jugar y calificación por edad según el estándar PEGI.

```
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="videojuegos.xsl"?>
<steam>
...

  <videojuego id="1">
    <titulo>Spiderman 2</titulo>
    <desarrollador>Insomniac Games</desarrollador>
    <lanzamiento>2023</lanzamiento>
    <genero>Acción</genero>
    <plataforma>PS5</plataforma>
    <calificacion>16</calificacion>
  </videojuego>

  <videojuego id="2">
    <titulo>Uncharted 4: El Legado del Ladrón</titulo>
    <desarrollador>Naughty Dog</desarrollador>
    <lanzamiento>2016</lanzamiento>
    <genero>Acción</genero>
    <plataforma>PS4 / PC</plataforma>
    <calificacion>16</calificacion>
  </videojuego>

  <videojuego id="3">
    <titulo>Minecraft</titulo>
    <desarrollador>Mojang Studios</desarrollador>
    <lanzamiento>2011</lanzamiento>
    <genero>Supervivencia</genero>
    <plataforma>Multiplataforma</plataforma>
    <calificacion>7</calificacion>
  </videojuego>
```

```
    <videojuego id="4">
      <titulo>Fortnite</titulo>
      <desarrollador>Epic Games</desarrollador>
      <lanzamiento>2017</lanzamiento>
      <genero>Battle Royale</genero>
      <plataforma>Multiplataforma</plataforma>
      <calificacion>12</calificacion>
    </videojuego>

    <videojuego id="5">
      <titulo>Valorant</titulo>
      <desarrollador>Riot Games</desarrollador>
      <lanzamiento>2020</lanzamiento>
      <genero>Shooter</genero>
      <plataforma>PC</plataforma>
      <calificacion>16</calificacion>
    </videojuego>
  </steam>
```

XSLT

Comenzamos con la transformación utilizando XSLT. Para ello, mi intención es transformar el código XML en una tabla, o incluso varias, para ver la información mejor clasificada y más ordenada.

1. Creación del Documento

El primer paso es crear el documento. En este caso, debe ser un archivo XSL y podrá tener el nombre que queramos. Es similar a XML, por lo que debe tener el mismo prólogo que un archivo XML normal.

```
1 <?xml version="1.0" encoding="UTF-8"?>
```

Si queremos que se vean los cambios reflejados, en el código XML debemos hacer la referencia a este nuevo archivo XSL.

```
2 <?xml-stylesheet type="text/xsl" href="videojuegos.xsl"?>
```

Tenemos que indicar que este documento será una hoja de estilos.

```
2 <xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">
```

Y finalmente, indicamos a qué formato queremos transformar el XML. En mi caso, HTML.

```
<xsl:output method="html" indent="yes"/>
```

2. Creación de la plantilla.

Lo siguiente que tenemos que hacer es crear la plantilla que se va a utilizar para dar el formato al código. Debemos usar `match="/"`, ya que queremos que la plantilla afecte a todo el código, desde la raíz. Cuando creamos la plantilla, abrimos las etiquetas `html` y en su interior, el `body` (de forma opcional podemos incluir el `head` si queremos añadir estilos).

```
<xsl:template match="/">
  <html>
    <head>
    </head>
    <body>
```

3. Añadir elementos HTML

En este punto solo debemos añadir etiquetas HTML, como si estuviésemos creando una página web. En los lugares correspondientes, debemos incluir las referencias a los elementos XML que queramos añadir.

En mi caso, he hecho 3 tablas, pero la forma de hacerlas es similar cambiando una pequeña opción, así que haré la explicación usando una de ellas como referencia.

Empezamos por el body, en el que he añadido un h1 a modo de título de la página y un h2 con el título de la tabla que le sigue. He creado una tabla (exactamente igual que en HTML) y la primera fila se hace igual, ya que es el encabezado.

```
<body>
  <h1>Lista de videojuegos</h1>

  <h2>Videojuegos multiplataforma</h2>
  <table>
    <tr style="background-color:orange">
      <th>Título</th>
      <th>Desarrollador</th>
      <th>Año de lanzamiento</th>
      <th>Género</th>
      <th>Plataforma</th>
      <th>Calificación</th>
      <th>Edad Mínima</th>
    </tr>
    <xsl:for-each select="steam/videojuego">
      <xsl:sort select="lanzamiento" order="descending"/>
      <xsl:if test="plataforma='Multiplataforma'">
        <tr>
          <td><xsl:value-of select="titulo"/></td>
          <td><xsl:value-of select="desarrollador"/></td>
          <td><xsl:value-of select="lanzamiento"/></td>
          <td><xsl:value-of select="genero"/></td>
          <td><xsl:value-of select="plataforma"/></td>
          <td><xsl:value-of select="calificacion"/></td>
          <xsl:choose>
            <xsl:when test="calificacion>=16">
              <td>Debes ser mayor de 16 años</td>
            </xsl:when>
            <xsl:otherwise>
              <td>Puedes jugar con cualquier edad</td>
            </xsl:otherwise>
          </xsl:choose>
        </tr>
      </xsl:if>
    </xsl:for-each>
  </table>
```

Una vez hecha la primera fila, he abierto una etiqueta **<xsl:for-each>** la cual sirve para hacer un bucle que mirará en todos los nodos que le indiquemos, en este caso, en cada videojuego. Este bucle se hace con el objetivo de que me añada a la tabla una fila por cada nodo videojuego que haya en el XML.

La siguiente etiqueta que se puede ver es **<xsl:sort>**, que consiste en modificar el orden en el que aparecen los nodos según las características de sus elementos. En mi caso, he querido que se ordenen según su año de lanzamiento, de más reciente a más antiguo.

Continuamos con la etiqueta **<xsl:if>**, la cual he utilizado para crear la condición y hacer las diferentes tablas con diferente información. Su funcionamiento es que comprueba todos los nodos que sean iguales a la condición que se le indica y esos son los que muestra. La he utilizado para crear tablas según el contenido de "plataforma".

La siguiente etiqueta y más necesaria es `<xsl:value-of>`. Esta etiqueta nos devuelve el valor que contiene el nodo que le indiquemos. Juntando esta etiqueta con el bucle (`<xsl:for-each>`), conseguimos que cada fila que se añada a la tabla, contenga información diferente a la anterior.

Finalmente, encontramos la etiqueta `<xsl:choose>`, con la que creamos una condición para que sea un resultado específico, o uno predeterminado. En este caso, la he utilizado junto a la etiqueta `<xsl:when>` para crear la condición específica, que se fija en el contenido del nodo calificación. El resultado predeterminado si no se cumple la condición, se establece con la etiqueta `<xsl:otherwise>`.

4. Añadir estilos “CSS”

Para que nuestra página quede más atractiva, podemos añadir estilos. No sé con exactitud si se utiliza CSS, pero sé que la sintaxis que se utiliza es la misma.

Para añadir estilos, debemos volver a la parte superior, en la que hemos abierto la etiqueta `<html>` y justo entre `<html>` y `<body>`, añadimos la etiqueta `<head>`. Dentro de esta etiqueta, abrimos `<style>` y aquí ya podremos escribir nuestros estilos utilizando el formato CSS.

En mi caso, he añadido bordes en la tabla para que se vean mejor los límites de las celdas.

También he añadido más espacio dentro de cada celda y he alineado su contenido al centro.

Y finalmente he añadido la función de que si se pasa el cursor por alguna fila, esta se destaque en otro color.

```
<style>
  table, th, tr, td{
    border: solid 1px black;
    border-collapse: collapse;
  }

  td{
    padding: 20px;
    text-align: center;
  }

  tr:hover{
    background-color: grey;
    color: white;
  }
</style>
```

5. Comprobamos resultados

Si ejecutamos nuestro archivo XML original con “Live Server” (en VSCode), podremos ver el resultado de nuestra transformación de un XML a un HTML.

Lista de videojuegos

Videojuegos multiplataforma

Título	Desarrollador	Año de lanzamiento	Género	Plataforma	Calificación	Edad Mínima
Fortnite	Epic Games	2017	Battle Royale	Multiplataforma	12	Puedes jugar con cualquier edad
Minecraft	Mojang Studios	2011	Supervivencia	Multiplataforma	7	Puedes jugar con cualquier edad

Videojuegos PC

Título	Desarrollador	Año de lanzamiento	Género	Plataforma	Calificación	Edad Mínima
Valorant	Riot Games	2020	Shooter	PC	16	Debes ser mayor de 16 años

Videojuegos Consola

Título	Desarrollador	Año de lanzamiento	Género	Plataforma	Calificación	Edad Mínima
Spiderman 2	Insomniac Games	2023	Acción	PS5	16	Debes ser mayor de 16 años
Uncharted 4: El Legado del Ladrón	Naughty Dog	2016	Acción	PS4 / PC	16	Debes ser mayor de 16 años

XSL-FO

Comenzamos con la transformación a PDF (XSL-FO). Para ello, mi intención es transformar el código XML inicial en un formato de impresión PDF para ver la información clasificada de forma limpia en un documento de texto.

1. Creación del documento

El primer paso es crear el documento. En este caso, debe ser un archivo XSL y podrá tener el nombre que queramos. Es similar a XML, por lo que debe tener el mismo prólogo que un archivo XML normal.

```
<?xml version="1.0" encoding="UTF-8"?>
```

Tenemos que indicar que este documento será una hoja de estilos, pero añadiendo la referencia a “FO” para que software sepa que vamos a dar formato a una página.

```
<xsl:stylesheet version="1.0"
xmlns:xsl="http://www.w3.org/1999/XSL/Transform"
xmlns:fo="http://www.w3.org/1999/XSL/Format">
```

Si queremos que se vean los cambios reflejados, en el código XML debemos hacer la referencia a este nuevo archivo XSL.

```
<?xml-stylesheet type="text/xsl" href="videojuegosPDF.xsl"?>
```

2. Creación de la plantilla

Lo siguiente que tenemos que hacer es crear la plantilla que se va a utilizar para dar el formato al código. Debemos usar `match="/"`, ya que queremos que la plantilla afecte a todo el código, desde la raíz.

Cuando creamos la plantilla para un PDF, en lugar de abrir etiquetas HTML, abrimos `<fo:root>` y `<fo:layout-master-set>`, donde establecemos los márgenes y el tamaño de la página.

```
<xsl:template match="/">
  <fo:root>
    <fo:layout-master-set>
      <fo:simple-page-master master-name="hoja" margin="1cm">
        <fo:region-body/>
      </fo:simple-page-master>
    </fo:layout-master-set>
  </fo:root>
</xsl:template>
```


3. Añadir elementos de formato

En este punto solo debemos añadir las etiquetas de FO, como si estuviéramos diseñando un folio en blanco. En los lugares correspondientes, debemos incluir las referencias a los elementos XML que queremos añadir.

```
<fo:page-sequence master-reference="hoja">
  <fo:flow flow-name="xsl-region-body">

    <xsl:for-each select="steam/videojuego">
      <fo:block font-weight="bold" font-size="14pt">
        <xsl:value-of select="titulo"/>
      </fo:block>

      <fo:block font-size="11pt">
        Desarrollador: <xsl:value-of select="desarrollador"/>
      </fo:block>
      <fo:block font-size="11pt">
        Lanzamiento: <xsl:value-of select="lanzamiento"/>
      </fo:block>
      <fo:block font-size="11pt">
        Género: <xsl:value-of select="genero"/>
      </fo:block>
      <fo:block font-size="11pt">
        Plataforma: <xsl:value-of select="plataforma"/>
      </fo:block>
      <fo:block font-size="11pt">
        Calificación: +<xsl:value-of select="calificacion"/>
      </fo:block>

      <fo:block border-bottom="1pt solid black" space-after="1em"/>
    </xsl:for-each>

  </fo:flow>
</fo:page-sequence>
```

En primer lugar se añade la etiqueta `<fo:page-sequence>`, que nos marca el inicio de la “página” sobre la que estamos actuando.

Seguimos con `<fo:flow>`, lo cual funciona como contenedor de la información que vamos a añadir, es decir, los datos que vamos a introducir.

Lo siguiente es la etiqueta `<xsl:for-each>`, que como hemos visto en XSLT, sirve para hacer un bucle que mira en todos los nodos que le indiquemos, en este caso, cada nodo videojuego.

A partir de aquí, cada elemento que queramos añadir se hará con un `<fo:block>`, lo que equivale a un párrafo en HTML.

Dentro de este bloque, podemos escribir el texto que queramos que aparezca y hacer referencia a los elementos del código XML usando la etiqueta `<xsl:value-of>`

Al terminar de mostrar cada nodo, he añadido un bloque con el estilo `border-bottom` para que se inserte una línea debajo de cada videojuego.

En el caso de querer poner estilos, habría que hacerlo añadiendo en cada bloque el estilo que queramos que aparezca.

4. Comprobamos resultados

Para comprobar los resultados, en este caso no es tan sencillo como ejecutarlo con “Live Server”, ya que necesitamos que nos genere el documento PDF.

Debemos descargarnos el zip de Apache FOP, el cual contiene el motor que nos transformará nuestro código XML siguiendo las instrucciones del XSL.

Cuando lo tenemos descargado y descomprimido, debemos dirigirnos a la carpeta donde se encuentre el archivo fop.bat y pegar los archivos XML y XSL.

fop.bat	24/04/2025 12:21	Archivo por lotes ...
fop.cmd	24/04/2025 12:21	Script de comand...
fop.js	24/04/2025 12:21	JSFile
videojuegos.xml	24/03/2026 18:28	Microsoft Edge H...
videojuegosPDF.xsl	24/03/2026 18:35	XSLT Stylesheet

Ahora damos clic derecho en un espacio vacío en la carpeta y la abrimos en el terminal.

Debemos ejecutar el siguiente comando, teniendo en cuenta los nombres de cada archivo.

```
.\fop.bat -xml videojuegos.xml -xsl videojuegosPDF.xsl -pdf videojuegos.pdf
```

Una vez ejecutado, se nos generará el documento PDF en la misma carpeta y podremos abrirlo.

